IN3005: Computer Graphics using OpenGL

Instructor: Dr. Eddie Edwards
Email: Philip.Edwards@city.ac.uk

Interim Coursework

Deliver by the due date shown on Moodle

Submission Details:

You must upload the finished coursework to the module submission site by the date set on Moodle. Standard lateness penalties and extensions policy applies. This assignment is scored out of 100 and scaled to **25% of the total marks for this module**.

You should submit a document (doc/pdf) that describes that provides written answers to the following questions. The % marks for each part are clearly shown. You will also need to submit code in the same manner as the final coursework. This will provide a trial run of the final submission process to iron out any issues that occur.

This submission will be via TurnitIn and will be fully checked for plagiarism. Please complete the task individually as evidence of copying will result in a zero mark and may result in disciplinary action.

Context:

As part of the main coursework, you will be asked to implement some “basic shapes”. The task for this interim coursework will be to implement one such shape. The choice of shape is yours to make. It could be as simple as a tetrahedron, diamond, cube or a more complex shape, such as a object with four walls and an angled roof. More complex shapes will score higher up to a point, but there’s no need to overdo it. This is a *basic* shape after all. The purpose of this coursework is to describe on paper how you would calculate the vertex positions for this object as well as normal vectors and other attributes for the vertices. Then you will need to design a suitable VBO to contain all the vertex properties. Starting from the Template code you will implement this shape in C++. To demonstrate understanding of camera positioning, you will create a circular camera path moving around your object. You will also create an OBJ file format version of your object. The shape created in this coursework can be used as one of the basic shapes needed for the final coursework. The creation of the circular track will also be instructive when you come to make your spline track in the final coursework.

CITY
UNIVERSITY OF LONDON
EST 1894

Task 1 – Object definition - geometry, normals, texture coordinates and triangles (15%)

Choosing your own dimensions for your object, draw a diagram showing how many vertices are required in the model and list the coordinates for all of the vertices. Choose a suitable origin and explain your choice.

Accurate rendering of the object will require correct normals. Normal vectors point outward from the surface. Calculate these normals for your design, describing your working as well as any changes that need to be made to the previous list of vertices to achieve this (hint: you may want to duplicate some vertices to allow each face to have consistent normals).

Texture coordinates define the corresponding position of the vertex in a texture image. Assign texture coordinates for each of your vertices and explain your choice.

List all the triangles that make up your object using indices to say which vertex is in which triangle. List the vertices in the correct order so that the winding of the triangle means that the face is pointing outward.

Task 2 – Vertex Buffer Object (VBO) design (15%)

You now have to design a VBO to contain the vertex attributes for your object. This should contain the normal, texture coordinates and colours for each vertex as vertex attributes. Describe a possible layout for the VBO and list in your report the calls to OpenGL from your client program required to transfer the VBO to the GPU. These should be native OpenGL commands. Do not assume that you have the helper classes from the template. Ensure that you give the correct values for the position and stride for each of the vertex attributes in VBO.

Task 3 – Implementation in the template code (20%)

In the Template you will see a section called basic shapes containing two objects - a plane and a sphere. Copy one of these (.h and .cpp) and rename both the file and the C++ class to represent your chosen object. You will need to reimplement the “Create” function to add the vertices, normals, texture coordinates and perhaps colour as well to your VBO design. This needs to be uploaded to the GPU and have the correct strides applied according to your answer to Task 2. The render function will draw the triangles and the release function deletes the VBO on the GPU. Having written this new class, include it in Game.cpp.

Task 4 – Shader implementation (20%)

What are the minimum shaders that are required to achieve rendering of the object? Provide the necessary GLSL code listings for these shaders to render the object at the correct location in

the scene. Include the transfer of any relevant information from the client program via uniform variables and the VBO. Write a new simple shader program to achieve this rendering, add it to the game in the `Game::Initialise() function` and render your object using the new shader instead of mainShader. If you replace the horse with your object it will be placed at the origin. Check that it looks as expected.

Task 5 – Camera positioning (10%)

You now want to place a camera so that it is looking at the object from a reasonable distance such that the object is a decent size the screen. Explain how terms *fovy, zNear, zFar* and *aspect* affect the viewing frustum when calling the function glm::perspective(). In the Game::Update() function in Game.cpp there is the following line (no 294 in the original file):

```
m_pCamera->Update(m_dt);
```

Comment or remove this line and instead use a call to

```
m_pCamera->Set(position, viewpoint, upVector)
```

You want the camera to move in a circular path such that it moves in around the object, with the object always being at the centre of the screen. The arguments to Set() are the same as those described for glm::lookAt(). You may want to be above the object looking down rather than across at ground level. Explain your working.

Following a similar pattern to the Path Starting Guide on Moodle, create a circular track on the ground plane with outer and inner points providing vertices in a triangle strip. The track should follow your circular path.

Task 6 - Texture mapping (10%)

Choose a suitable texture image and add this to your object. Check that the appearance is as you would expect when running the game. Also add a suitable texture to your track.

Task 7 – OBJ Format (10%)

An OBJ file is a simple text format for listing vertex positions, normals and texture coordinates. Looking at the horse example, we have:

```
File: horse2.obj:
----------------------------
# (comment lines)
mtllib horse2.mtl                    Material file
#
#  Mesh
#
g  Mesh


#
#  Mesh_Material.001
#
usemtl  Mesh_Material.001            Name of material to use

v 0.868769 4.436345 0.464442         Vertex Positions
v 0.917819 4.410617 0.000000
v 0.394835 4.320202 0.000000
.
.
.
vn -0.239304 0.863308 0.444335       Vertex Normals
vn -0.220471 0.975394 0.000000
vn -0.193615 0.981078 0.000000
.
.
.
vt 0.547347 0.409156                 Texture coordinates
vt 0.535746 0.473269
vt 0.462320 0.449056
.
.
.
f 1/1/1 2/2/2 3/3/3                  Faces (indices of v/vt/vn)
f 1/1/1 3/3/3 4/4/4
f 3/3/3 2/2/2 5/1/5
```

File: horse2.mtl:
---
newmtl Mesh_Material.001 Name of material
Ns 86.274510
Ka 0.000000 0.000000 0.000000
Kd 0.049547 0.023813 0.008941 Surface lighting properties
Ks 0.030000 0.027306 0.013816
Ni 1.000000
d 0.000000
illum 2
map_Kd horse.bmp Name of associated texture image

Write an OBJ/MTL file for your object and include this in your submission.

Deliverables:

ZIP file (note this must be less than 200MB!) uploaded to Moodle containing:

1. Your report (answers to tasks 1 and 2 and descriptions of the other tasks) in PDF/Word format.
2. A folder containing the source code and the OBJ file.
    - The code must be at a stage where a simple 'clean, build & execute' will start your application. Code that does not compile and execute will receive a **FAIL**.
    - Do not include the .db file or the .vs directory– this can be regenerated automatically by the compiler and can be very large.
    - Run Build->Clean Solution before zipping and submitting. This will save space.

Academic Misconduct/Plagiarism:

If you copy the work of others (either that of another student/team or of a third party), with or without their
permission, you will score no marks and further disciplinary action will be taken against you. The same
applies if you allow others to copy your work. This is a group responsibility.

Sample code (labs) available on Moodle as term progresses. Study and use of sample code from the
internet is encouraged but must be referenced if you include it in your work.

Use of AI, e.g. ChatGPT, Bard, Gemini, DeepSeek: using these aids without acknowledgement is
academic misconduct. You are free to use such aids if you think it useful, IF AND ONLY IF you add an
appendix (there is no limit on its length) in which you say exactly how you used them: sequences of
prompt used and outputs received, how you used these outputs.